Computer Architecture

Project: MIPS 32 Processor Implementation

Single-Cycle and Pipelined Architectures

Problem: Sum of powers of 2 within an interval $[a, b]$

Student:
Măgerușan Alexandru

Date:
May 24, 2026

Technical University of Cluj-Napoca
Faculty of Automation and Computer Science

Contents

1 Introduction

This project presents the hardware design of a 32-bit MIPS microprocessor implemented in VHDL. The implementation is divided into two distinct evolutionary phases:

- **Phase 1:** A foundational Single-Cycle architecture.
- **Phase 2:** An advanced 5-stage Pipelined architecture with hazard resolution.

The system is validated by solving a specific problem: calculating the sum of powers of 2 from a given dataset, conditionally restricted to a specified interval.

2 Solution Algorithm

2.1 C++ Source Code

High-level reference implementation of the algorithm.

```cpp
#include <stdio.h>

int main()
{
    int a = 2;
    int b = 90;
    int x[] = {0, 1, 2, 3, 256, 14, 16};
    int n = 7;

    int sum = 0;
    int i = -1;

    while (i < n - 1)
    {
        i++;
        int val = x[i];

        if (val < a) continue;
        if (val > b) continue;
        if (val <= 0) continue;

        int temp = val - 1;
        int test = val & temp;

        if (test != 0) continue;

        sum = sum + val;
    }

    printf("Sum:%d\n", sum);
    return 0;
}
```

C++ Reference Program

2.2 x86 Assembly Code

Generated x86 code for testing the processing logic.

```
main:
        push    rbp
        mov     rbp, rsp
        mov     DWORD PTR [rbp-12], 2
        mov     DWORD PTR [rbp-16], 90
        mov     DWORD PTR [rbp-64], 0
        mov     DWORD PTR [rbp-60], 1
        mov     DWORD PTR [rbp-56], 2
        mov     DWORD PTR [rbp-52], 3
        mov     DWORD PTR [rbp-48], 256
        mov     DWORD PTR [rbp-44], 14
        mov     DWORD PTR [rbp-40], 16
        mov     DWORD PTR [rbp-20], 7
        mov     DWORD PTR [rbp-4], 0
        mov     DWORD PTR [rbp-8], -1
        jmp     .L2
.L7:
        add     DWORD PTR [rbp-8], 1
        mov     eax, DWORD PTR [rbp-8]
        cdqe
        mov     eax, DWORD PTR [rbp-64+rax*4]
        mov     DWORD PTR [rbp-24], eax
        mov     eax, DWORD PTR [rbp-24]
        cmp     eax, DWORD PTR [rbp-12]
        jge     .L3
        jmp     .L2
.L3:
        mov     eax, DWORD PTR [rbp-24]
        cmp     eax, DWORD PTR [rbp-16]
        jle     .L4
        jmp     .L2
.L4:
        cmp     DWORD PTR [rbp-24], 0
        jg      .L5
        jmp     .L2
.L5:
        mov     eax, DWORD PTR [rbp-24]
        sub     eax, 1
        mov     DWORD PTR [rbp-28], eax
        mov     eax, DWORD PTR [rbp-24]
        and     eax, DWORD PTR [rbp-28]
        mov     DWORD PTR [rbp-32], eax
        cmp     DWORD PTR [rbp-32], 0
        je      .L6
        jmp     .L2
.L6:
        mov     eax, DWORD PTR [rbp-24]
        add     DWORD PTR [rbp-4], eax
.L2:
        mov     eax, DWORD PTR [rbp-20]
        sub     eax, 1
        cmp     DWORD PTR [rbp-8], eax
        jl      .L7
        mov     eax, 0
        pop     rbp
        ret
```

x86 Assembly Code

2.3 MIPS Assembly Code

Final implementation intended for execution on the MIPS processor.

```
.data
    # Declare the array in memory (each number occupies 1 word = 4 bytes)
    x: .word 0, 1, 2, 3, 256, 14, 16

.text
.globl main
main:
    addi $1, $0, 2      # a = 2
    addi $2, $0, 90     # b = 90
    addi $3, $0, 7      # n = 7
    la   $4, x          # base_addr = starting address of array x
    add  $5, $0, $0     # sum = 0
    add  $6, $0, $0     # i = 0

loop_start:
    slt  $7, $6, $3     # $7 = 1 if i < n
    beq  $7, $0, end    # If i >= n, done, jump to 'end'

    # --- ADDRESS CORRECTION ---
    sll  $9, $6, 2      # $9 = i * 4
    add  $9, $4, $9     # $9 = address of current element
    lw   $8, 0($9)      # val = read x[i] into $8

    addi $6, $6, 1      # i = i + 1

    # Check: val < a
    slt  $7, $8, $1     # $7 = 1 if val < a
    bne  $7, $0, loop_start

    # Check: val > b
    slt  $7, $2, $8     # $7 = 1 if b < val
    bne  $7, $0, loop_start

    # Check: val > 0
    slt  $7, $0, $8     # $7 = 1 if 0 < val
    beq  $7, $0, loop_start

    # Check: is it a power of 2? (val & (val - 1) == 0)
    addi $9, $8, -1     # $9 = val - 1
    and  $9, $8, $9     # $9 = val & (val - 1)
    bne  $9, $0, loop_start

    # Add to sum
    add  $5, $5, $8     # sum = sum + val
    j    loop_start

end:
    addi $v0, $0, 10    # exit
    syscall
```

MIPS Assembly Code for the processor

3 Machine Code and ROM Mapping

The following table details the binary and hexadecimal translation of the MIPS Assembly code, as inserted into the instruction memory (`ROM`) for both architectures.

PC	Machine Code (Binary)	Hex	MIPS ASM
0	001000_00000_00001_0000000000000010	20010002	`addi $1, $0, 2`
1	001000_00000_00010_0000000001011010	2002005A	`addi $2, $0, 90`
2	001000_00000_00011_0000000000000111	20030007	`addi $3, $0, 7`
3	000000_00000_00000_00100_00000_100000	00002020	`add $4, $0, $0` *(la $4, x)*
4	000000_00000_00000_00101_00000_100000	00002820	`add $5, $0, $0`
5	000000_00000_00000_00110_00000_100000	00003020	`add $6, $0, $0`
— loop_start: (PC = 6) —			
6	000000_00110_00011_00111_00000_101010	00C3382A	`slt $7, $6, $3`
7	000100_00111_00000_0000000000001111	10E0000F	`beq $7, $0, end` *(+15)*
8	000000_00000_00110_01001_00010_000000	00064880	`sll $9, $6, 2`
9	000000_00100_01001_01001_00000_100000	00894820	`add $9, $4, $9`
10	100011_01001_01000_0000000000000000	8D280000	`lw $8, 0($9)`
11	001000_00110_00110_0000000000000001	20C60001	`addi $6, $6, 1`
12	000000_01000_00001_00111_00000_101010	0101382A	`slt $7, $8, $1`
13	000101_00111_00000_1111111111111000	14E0FFF8	`bne $7, $0, loop_start` *(-8)*
14	000000_00010_01000_00111_00000_101010	0048382A	`slt $7, $2, $8`
15	000101_00111_00000_1111111111110110	14E0FFF6	`bne $7, $0, loop_start` *(-10)*
16	000000_00000_01000_00111_00000_101010	0008382A	`slt $7, $0, $8`
17	000100_00111_00000_1111111111110100	10E0FFF4	`beq $7, $0, loop_start` *(-12)*
18	001000_01000_01001_1111111111111111	2109FFFF	`addi $9, $8, -1`
19	000000_01000_01001_01001_00000_100100	01094824	`and $9, $8, $9`
20	000101_01001_00000_1111111111110001	1520FFF1	`bne $9, $0, loop_start` *(-15)*
21	000000_00101_01000_00101_00000_100000	00A82820	`add $5, $5, $8`
22	000010_00000000000000000000000110	08000006	`j loop_start` *(Jmp target 6)*
— end: (PC = 23) —			
23	001000_00000_00010_0000000000001010	2002000A	`addi $v0, $0, 10` *($v0 is $2)*
24	000000_00000_00000_00000_00000_001100	0000000C	`syscall` *(generic syscall code)*

Table 1: Detailed mapping between source code and ROM memory

Part I
Single-Cycle Implementation

This part details the initial implementation of the processor, where each instruction executes completely within a single clock cycle.

4 Instruction Set and Control Signals

The processor decodes a subset of 10 MIPS 32 instructions. The following table presents the binary encodings and signals generated by the Control Unit (`UC`).

Instruction	Opcode [31-26]	RegDst	ExtOp	ALUSrc	Branch	Br_ne	Jump	MemWrite	MemtoReg	RegWrite	ALUOp[1:0]	function [5-0]	ALUCtrl[2:0]
`add`	000000	1	0	0	0	0	0	0	0	1	10	100000	000 (+)
`and`	000000	1	0	0	0	0	0	0	0	1	10	100100	010 (&)
`slt`	000000	1	0	0	0	0	0	0	0	1	10	101010	011 (<)
`sll`	000000	1	0	0	0	0	0	0	0	1	10	000000	100 (<<)
`addi`	001000	0	1	1	0	0	0	0	0	1	00 (+)	X	000 (+)
`lw`	100011	0	1	1	0	0	0	0	1	1	00 (+)	X	000 (+)
`sw`	101011	0	1	1	0	0	0	1	0	0	00 (+)	X	000 (+)
`beq`	000100	0	1	0	1	0	0	0	0	0	01 (-)	X	001 (-)
`bne`	000101	0	1	0	0	1	0	0	0	0	01 (-)	X	001 (-)
`j`	000010	0	0	0	0	0	1	0	0	0	00	X	000

Table 2: Instruction encodings and control signals

4.1 Instruction Classification and Datapath

4.1.1 R-Type Instructions (Register)

Uses three registers: sources `rs`, `rt` and destination `rd`. The ALU performs the operation indicated by `funct`.

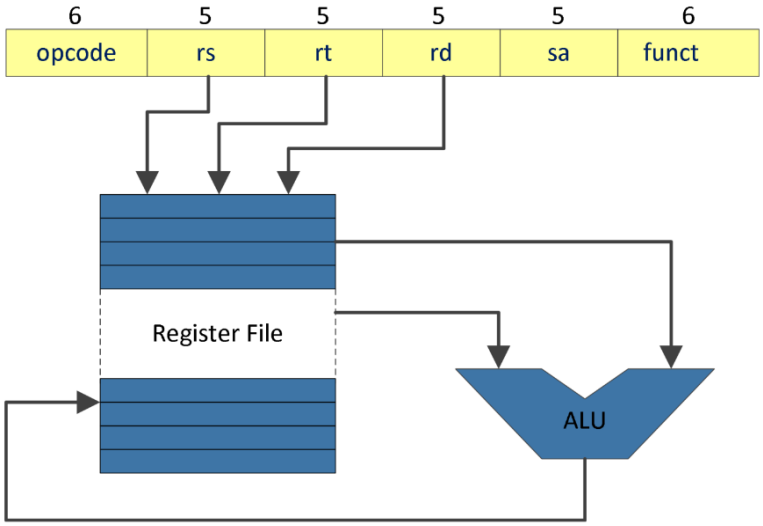

Figure 1: Datapath for R-Type instructions

4.1.2 I-Type Instructions (Immediate)

Uses a 16-bit immediate constant, extended to 32 bits (sign or zero) before entering the ALU.

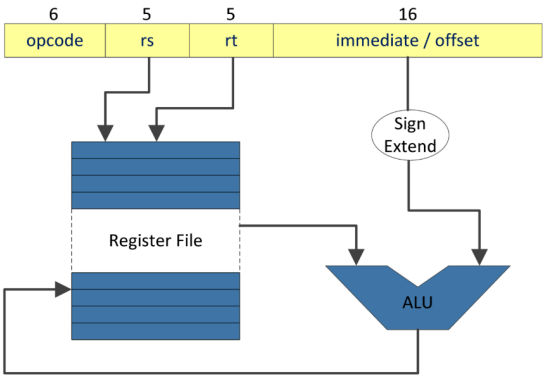

Figure 2: Sign Extend (`addi`)

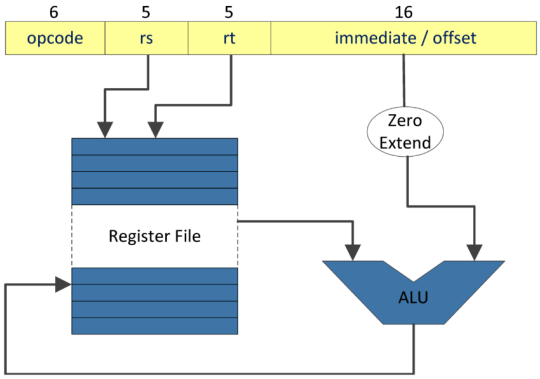

Figure 3: Zero Extend

Memory Access (`lw` and `sw`) The memory address is the sum of the base register and the sign-extended offset.

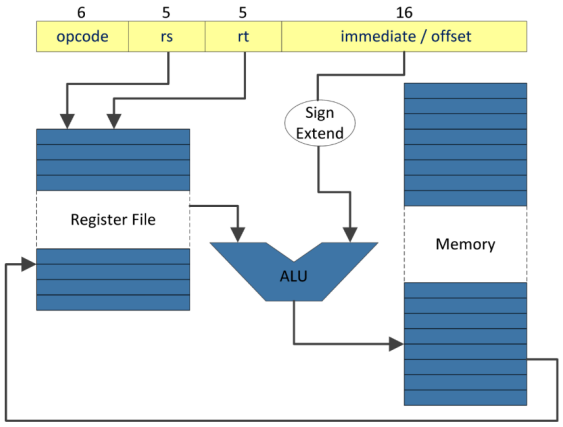

Figure 4: Load (`lw`)

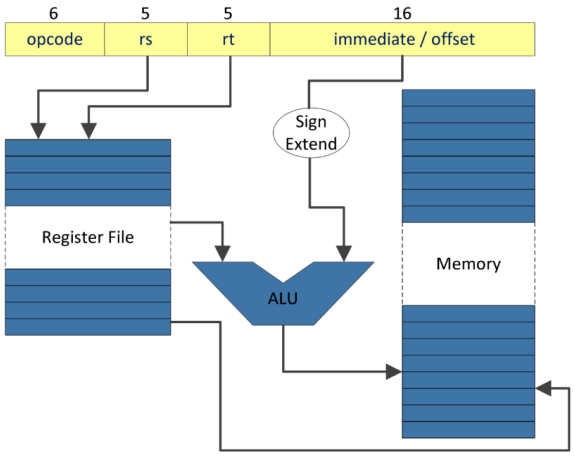

Figure 5: Store (`sw`)

Conditional Branch (`beq`, `bne`) The branch decision is based on checking operand equality in the ALU.

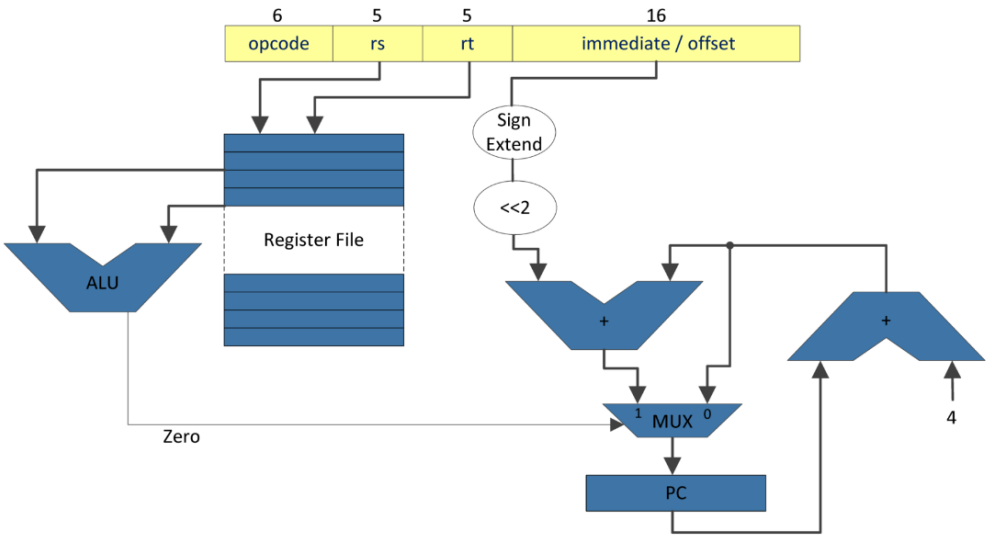

Figure 6: Branch instructions

4.1.3 J-Type Instructions (Jump)

Directly modifies the Program Counter with the provided address.

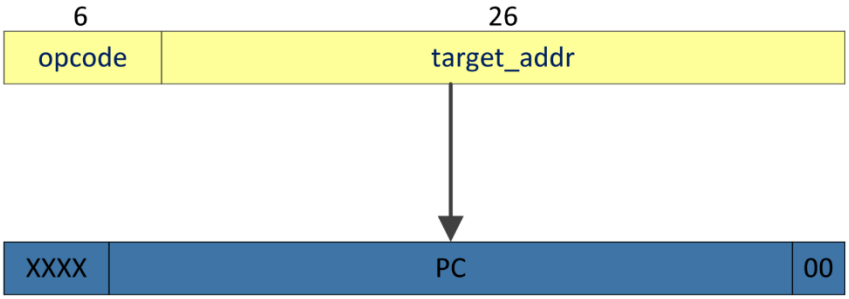

Figure 7: Jump instructions

5 Hardware Components and Architecture

5.1 Processor Block Diagram

The diagram illustrates the interconnection of the IFetch, IDecode, EX, and MEM modules. It was logically extended to support the `bne` instruction.

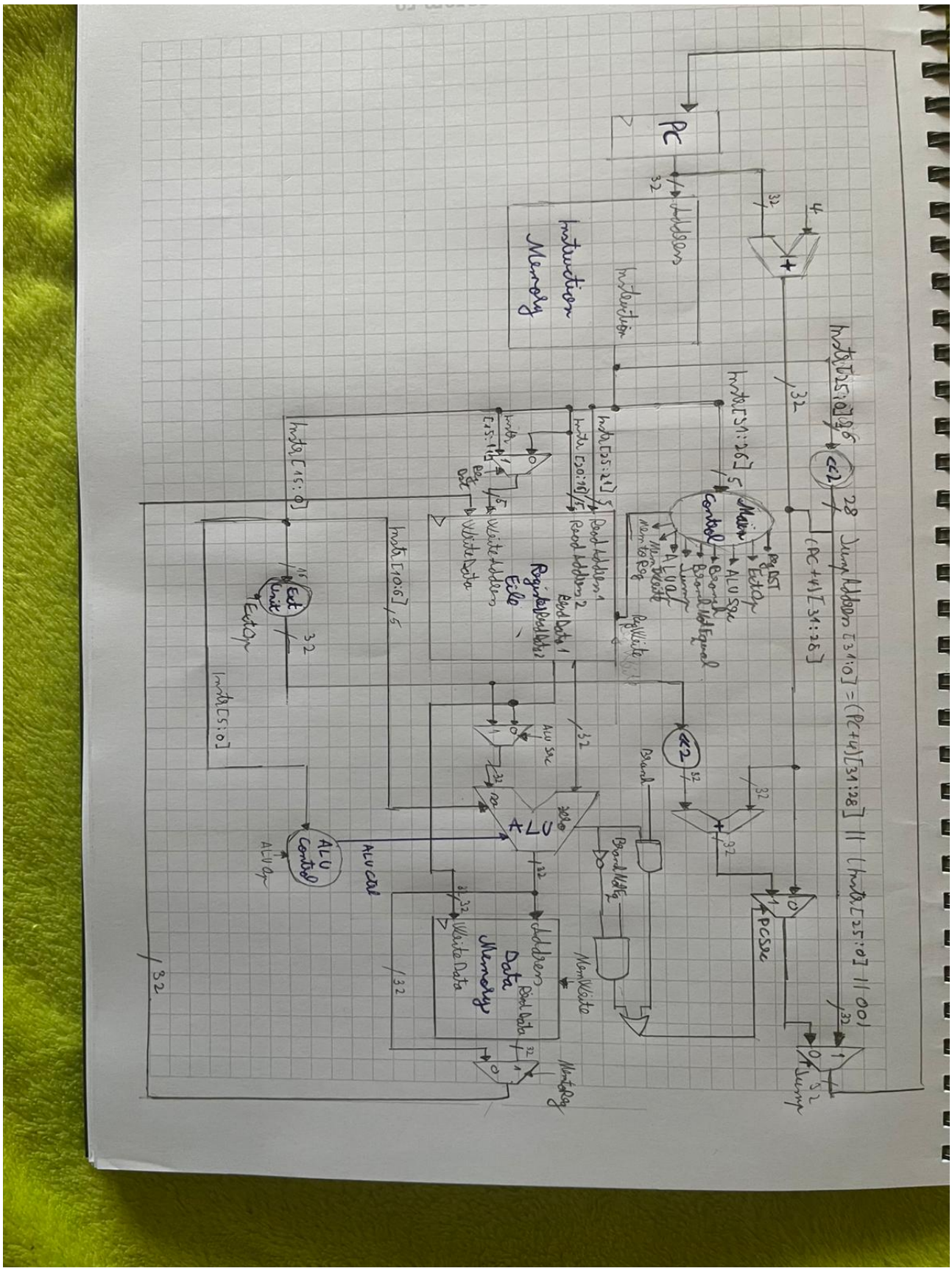

Figure 8: MIPS Single-Cycle Architecture

5.2 IFetch.vhd

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity IFetch is
    Port ( clk            : in  STD_LOGIC;
           rst            : in  STD_LOGIC;
           en             : in  STD_LOGIC;
           BranchAddress  : in  STD_LOGIC_VECTOR (31 downto 0);
           JumpAddress    : in  STD_LOGIC_VECTOR (31 downto 0);
           Jump           : in  STD_LOGIC;
           PCSrc          : in  STD_LOGIC;
           Instruction    : out STD_LOGIC_VECTOR (31 downto 0);
           PC_plus_4      : out STD_LOGIC_VECTOR (31 downto 0));
end IFetch;

architecture Behavioral of IFetch is
    signal PC      : STD_LOGIC_VECTOR(31 downto 0) := (others => '0');
    signal Next_PC : STD_LOGIC_VECTOR(31 downto 0);
    signal PC4     : STD_LOGIC_VECTOR(31 downto 0);
    type rom_type is array (0 to 31) of std_logic_vector(31 downto 0);
    signal ROM : rom_type := (
        0  => B"000000_00000_00000_00100_00000_100000",
        1  => B"001000_00000_01010_0000000000010000",
        2  => B"100011_00100_00001_0000000000000000",
        3  => B"100011_00100_00010_0000000000000100",
        4  => B"100011_00100_00011_0000000000001000",
        5  => B"000000_00000_00000_00101_00000_100000",
        6  => B"000000_00000_00000_00110_00000_100000",
        7  => B"000000_00110_00011_00111_00000_101010",
        8  => B"000100_00111_00000_0000000000001111",
        9  => B"000000_00000_00110_01001_00010_000000",
        10 => B"000000_01010_01001_01001_00000_100000",
        11 => B"100011_01001_01000_0000000000000000",
        12 => B"001000_00110_00110_0000000000000001",
        13 => B"000000_01000_00001_00111_00000_101010",
        14 => B"000101_00111_00000_1111111111111000",
        15 => B"000000_00010_01000_00111_00000_101010",
        16 => B"000101_00111_00000_1111111111110110",
        17 => B"000000_00000_01000_00111_00000_101010",
        18 => B"000100_00111_00000_1111111111110100",
        19 => B"001000_01000_01001_1111111111111111",
        20 => B"000000_01000_01001_01001_00000_100100",
        21 => B"000101_01001_00000_1111111111110001",
        22 => B"000000_00101_01000_00101_00000_100000",
        23 => B"000010_00000000000000000000000111",
        24 => B"101011_00100_00101_0000000000001100",
        25 => B"000010_00000000000000000000011001",
        others => B"000000_00000_00000_00000_00000_000000"
    );
begin
    process(clk, rst)
    begin
        if rst = '1' then
            PC <= (others => '0');
        elsif rising_edge(clk) then
            if en = '1' then
                PC <= Next_PC;
            end if;
        end if;
```

```
    end process;

    PC4 <= PC + 4;
    PC_plus_4 <= PC4;
    Next_PC <= JumpAddress when Jump = '1' else
               BranchAddress when PCSrc = '1' else
               PC4;
    Instruction <= ROM(conv_integer(PC(6 downto 2)));
end Behavioral;
```

5.3 IDecode.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity IDecode is
    Port ( clk          : in  STD_LOGIC;
           en           : in  STD_LOGIC;
           Instr        : in  STD_LOGIC_VECTOR (25 downto 0);
           WD           : in  STD_LOGIC_VECTOR (31 downto 0);
           RegWrite     : in  STD_LOGIC;
           RegDst       : in  STD_LOGIC;
           ExtOp        : in  STD_LOGIC;
           RD1          : out STD_LOGIC_VECTOR (31 downto 0);
           RD2          : out STD_LOGIC_VECTOR (31 downto 0);
           Ext_Imm      : out STD_LOGIC_VECTOR (31 downto 0);
           func         : out STD_LOGIC_VECTOR (5 downto 0);
           sa           : out STD_LOGIC_VECTOR (4 downto 0));
end IDecode;

architecture Behavioral of IDecode is
    type reg_array is array(0 to 31) of STD_LOGIC_VECTOR(31 downto 0);
    signal reg_file : reg_array := (others => x"00000000");
    signal WriteAddress : STD_LOGIC_VECTOR(4 downto 0);
begin
    WriteAddress <= Instr(15 downto 11) when RegDst = '1' else Instr(20
       downto 16);
    process(clk)
    begin
        if rising_edge(clk) then
            if en = '1' and RegWrite = '1' then
                reg_file(conv_integer(WriteAddress)) <= WD;
            end if;
        end if;
    end process;

    RD1 <= reg_file(conv_integer(Instr(25 downto 21)));
    RD2 <= reg_file(conv_integer(Instr(20 downto 16)));
    Ext_Imm(15 downto 0) <= Instr(15 downto 0);
    Ext_Imm(31 downto 16) <= (others => Instr(15)) when ExtOp = '1' else (
       others => '0');
    func <= Instr(5 downto 0);
    sa   <= Instr(10 downto 6);
end Behavioral;
```

5.4 EX.vhd

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;

entity EX is
    Port ( PC_plus_4     : in  STD_LOGIC_VECTOR (31 downto 0);
           RD1           : in  STD_LOGIC_VECTOR (31 downto 0);
           RD2           : in  STD_LOGIC_VECTOR (31 downto 0);
           Ext_Imm       : in  STD_LOGIC_VECTOR (31 downto 0);
           ALUSrc        : in  STD_LOGIC;
           ALUOp         : in  STD_LOGIC_VECTOR (1 downto 0);
           func          : in  STD_LOGIC_VECTOR (5 downto 0);
           sa            : in  STD_LOGIC_VECTOR (4 downto 0);
           BranchAddress : out STD_LOGIC_VECTOR (31 downto 0);
           ALURes        : out STD_LOGIC_VECTOR (31 downto 0);
           Zero          : out STD_LOGIC);
end EX;

architecture Behavioral of EX is
    signal ALUCtrl : STD_LOGIC_VECTOR(2 downto 0);
    signal A, B    : STD_LOGIC_VECTOR(31 downto 0);
    signal ALU_Out : STD_LOGIC_VECTOR(31 downto 0);
begin
    BranchAddress <= PC_plus_4 + (Ext_Imm(29 downto 0) & "00");
    A <= RD1;
    B <= Ext_Imm when ALUSrc = '1' else RD2;

    process(ALUOp, func)
    begin
        case ALUOp is
            when "00" => ALUCtrl <= "000";
            when "01" => ALUCtrl <= "001";
            when "10" =>
                case func is
                    when "100000" => ALUCtrl <= "000";
                    when "100100" => ALUCtrl <= "010";
                    when "101010" => ALUCtrl <= "011";
                    when "000000" => ALUCtrl <= "100";
                    when others   => ALUCtrl <= "000";
                end case;
            when others => ALUCtrl <= "000";
        end case;
    end process;

    process(A, B, ALUCtrl, sa)
    begin
        case ALUCtrl is
            when "000" => ALU_Out <= A + B;
            when "001" => ALU_Out <= A - B;
            when "010" => ALU_Out <= A and B;
            when "011" =>
                if signed(A) < signed(B) then ALU_Out <= x"00000001";
                else ALU_Out <= x"00000000"; end if;
            when "100" => ALU_Out <= to_stdlogicvector(to_bitvector(B) sll
               conv_integer(sa));
            when others => ALU_Out <= (others => '0');
        end case;
    end process;

```

```
    ALURes <= ALU_Out;
    Zero <= '1' when ALU_Out = x"00000000" else '0';
end Behavioral;
```

5.5 MEM.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity MEM is
    Port ( clk         : in  STD_LOGIC;
           en          : in  STD_LOGIC;
           MemWrite    : in  STD_LOGIC;
           ALUResin    : in  STD_LOGIC_VECTOR (31 downto 0);
           RD2         : in  STD_LOGIC_VECTOR (31 downto 0);
           MemData     : out STD_LOGIC_VECTOR (31 downto 0);
           ALUResout   : out STD_LOGIC_VECTOR (31 downto 0));
end MEM;

architecture Behavioral of MEM is
    type ram_type is array (0 to 63) of STD_LOGIC_VECTOR(31 downto 0);
    signal RAM : ram_type := (
        0 => x"00000002", -- RAM[0]: a = 2
        1 => x"0000005A", -- RAM[4]: b = 90
        2 => x"00000007", -- RAM[8]: n = 7
        3 => x"00000000", -- RAM[12]: sum = 0
        4 => x"00000000", -- RAM[16]: x[0] = 0
        5 => x"00000001", -- RAM[20]: x[1] = 1
        6 => x"00000002", -- RAM[24]: x[2] = 2
        7 => x"00000003", -- RAM[28]: x[3] = 3
        8 => x"00000100", -- RAM[32]: x[4] = 256
        9 => x"0000000E", -- RAM[36]: x[5] = 14
        10 => x"00000010", -- RAM[40]: x[6] = 16
        others => x"00000000"
    );

begin
    process(clk)
    begin
        if rising_edge(clk) then
            if en = '1' and MemWrite = '1' then
                RAM(conv_integer(ALUResin(7 downto 2))) <= RD2;
            end if;
        end if;
    end process;
    MemData <= RAM(conv_integer(ALUResin(7 downto 2)));
    ALUResout <= ALUResin;
end Behavioral;
```

5.6 UC.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity UC is
    Port ( Instr_opcode : in  STD_LOGIC_VECTOR (5 downto 0);
           RegDst       : out STD_LOGIC;
```

```
            ExtOp         : out STD_LOGIC;
            ALUSrc        : out STD_LOGIC;
            Branch        : out STD_LOGIC;
            BranchNotEq   : out STD_LOGIC;
            Jump          : out STD_LOGIC;
            ALUOp         : out STD_LOGIC_VECTOR (1 downto 0);
            MemWrite      : out STD_LOGIC;
            MemtoReg      : out STD_LOGIC;
            RegWrite      : out STD_LOGIC);
end UC;

architecture Behavioral of UC is
begin
    process(Instr_opcode)
    begin
        RegDst <= '0'; ExtOp <= '0'; ALUSrc <= '0'; Branch <= '0';
        BranchNotEq <= '0'; Jump <= '0'; MemWrite <= '0'; MemtoReg <= '0';
        RegWrite <= '0'; ALUOp <= "00";

        case Instr_opcode is
            when "000000" => RegDst <= '1'; RegWrite <= '1'; ALUOp <= "10";
            when "001000" => ExtOp <= '1'; ALUSrc <= '1'; RegWrite <= '1';
               ALUOp <= "00";
            when "100011" => ExtOp <= '1'; ALUSrc <= '1'; MemtoReg <= '1';
               RegWrite <= '1'; ALUOp <= "00";
            when "101011" => ExtOp <= '1'; ALUSrc <= '1'; MemWrite <= '1';
               ALUOp <= "00";
            when "000100" => ExtOp <= '1'; Branch <= '1'; ALUOp <= "01";
            when "000101" => ExtOp <= '1'; BranchNotEq <= '1'; ALUOp <= "01"
               ;
            when "000010" => Jump <= '1';
            when others => null;
        end case;
    end process;
end Behavioral;
```

5.7 test_env.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity test_env is
    Port ( clk : in  STD_LOGIC;
           btn : in  STD_LOGIC_VECTOR (4 downto 0);
           sw  : in  STD_LOGIC_VECTOR (15 downto 0);
           led : out STD_LOGIC_VECTOR (15 downto 0);
           an  : out STD_LOGIC_VECTOR (7 downto 0);
           cat : out STD_LOGIC_VECTOR (6 downto 0));
end test_env;

architecture Behavioral of test_env is
    component IFetch is
        Port ( clk, rst, en : in STD_LOGIC;
               BranchAddress, JumpAddress : in STD_LOGIC_VECTOR (31 downto
                  0);
               Jump, PCSrc : in STD_LOGIC;
               Instruction, PC_plus_4 : out STD_LOGIC_VECTOR (31 downto 0));
    end component;
```

```
21
22      component UC is
23          Port ( Instr_opcode : in STD_LOGIC_VECTOR (5 downto 0);
24                 RegDst, ExtOp, ALUSrc, Branch, BranchNotEq, Jump : out
                      STD_LOGIC;
25                 ALUOp : out STD_LOGIC_VECTOR (1 downto 0);
26                 MemWrite, MemtoReg, RegWrite : out STD_LOGIC);
27      end component;
28
29      component IDecode is
30          Port ( clk, en : in STD_LOGIC;
31                 Instr : in STD_LOGIC_VECTOR (25 downto 0);
32                 WD : in STD_LOGIC_VECTOR (31 downto 0);
33                 RegWrite, RegDst, ExtOp : in STD_LOGIC;
34                 RD1, RD2, Ext_Imm : out STD_LOGIC_VECTOR (31 downto 0);
35                 func : out STD_LOGIC_VECTOR (5 downto 0);
36                 sa : out STD_LOGIC_VECTOR (4 downto 0));
37      end component;
38
39      component EX is
40          Port ( PC_plus_4, RD1, RD2, Ext_Imm : in STD_LOGIC_VECTOR (31 downto
                0);
41                 ALUSrc : in STD_LOGIC;
42                 ALUOp : in STD_LOGIC_VECTOR (1 downto 0);
43                 func : in STD_LOGIC_VECTOR (5 downto 0);
44                 sa : in STD_LOGIC_VECTOR (4 downto 0);
45                 BranchAddress, ALURes : out STD_LOGIC_VECTOR (31 downto 0);
46                 Zero : out STD_LOGIC);
47      end component;
48
49      component MEM is
50          Port ( clk, en, MemWrite : in STD_LOGIC;
51                 ALUResin, RD2 : in STD_LOGIC_VECTOR (31 downto 0);
52                 MemData, ALUResout : out STD_LOGIC_VECTOR (31 downto 0));
53      end component;
54
55      component MPG is
56          Port ( enable : out STD_LOGIC;
57                 btn, clk : in STD_LOGIC);
58      end component;
59
60      component SSD is
61          Port ( clk : in STD_LOGIC;
62                 digits : in STD_LOGIC_VECTOR(31 downto 0);
63                 an : out STD_LOGIC_VECTOR(7 downto 0);
64                 cat : out STD_LOGIC_VECTOR(6 downto 0));
65      end component;
66
67      signal en, rst : STD_LOGIC;
68      signal Instruction, PC_plus_4 : STD_LOGIC_VECTOR(31 downto 0);
69      signal RegDst, ExtOp, ALUSrc, Branch, BranchNotEq, Jump, MemWrite,
           MemtoReg, RegWrite : STD_LOGIC;
70      signal ALUOp : STD_LOGIC_VECTOR(1 downto 0);
71      signal RD1, RD2, Ext_Imm : STD_LOGIC_VECTOR(31 downto 0);
72      signal func : STD_LOGIC_VECTOR(5 downto 0);
73      signal sa : STD_LOGIC_VECTOR(4 downto 0);
74      signal BranchAddress, ALURes, MemData, ALUResout, WD, JumpAddress :
           STD_LOGIC_VECTOR(31 downto 0);
75      signal Zero, PCSrc : STD_LOGIC;
76      signal digits_out : STD_LOGIC_VECTOR(31 downto 0);
```

```
77
78  begin
79      empege: MPG port map (enable => en, btn => btn(0), clk => clk);
80      rst <= btn(1);
81      display: SSD port map (clk => clk, digits => digits_out, an => an, cat
            => cat);
82
83      inst_IFetch: IFetch port map (clk, rst, en, BranchAddress, JumpAddress,
            Jump, PCSrc, Instruction, PC_plus_4);
84      inst_UC: UC port map (Instruction(31 downto 26), RegDst, ExtOp, ALUSrc,
            Branch, BranchNotEq, Jump, ALUOp, MemWrite, MemtoReg, RegWrite);
85      inst_IDecode: IDecode port map (clk, en, Instruction(25 downto 0), WD,
            RegWrite, RegDst, ExtOp, RD1, RD2, Ext_Imm, func, sa);
86      inst_EX: EX port map (PC_plus_4, RD1, RD2, Ext_Imm, ALUSrc, ALUOp, func,
             sa, BranchAddress, ALURes, Zero);
87      inst_MEM: MEM port map (clk, en, MemWrite, ALURes, RD2, MemData,
            ALUResout);
88
89      WD <= MemData when MemtoReg = '1' else ALUResout;
90      PCSrc <= (Branch and Zero) or (BranchNotEq and not Zero);
91      JumpAddress <= PC_plus_4(31 downto 28) & Instruction(25 downto 0) & "00"
            ;
92
93      led(0) <= RegDst; led(1) <= ExtOp; led(2) <= ALUSrc; led(3) <= Branch;
94      led(4) <= BranchNotEq; led(5) <= Jump; led(6) <= MemWrite; led(7) <=
            MemtoReg;
95      led(8) <= RegWrite; led(10 downto 9) <= ALUOp; led(15 downto 11) <= (
            others => '0');
96
97      process(sw(7 downto 5), Instruction, PC_plus_4, RD1, RD2, Ext_Imm,
            ALURes, MemData, WD)
98      begin
99          case sw(7 downto 5) is
100             when "000" => digits_out <= Instruction;
101             when "001" => digits_out <= PC_plus_4;
102             when "010" => digits_out <= RD1;
103             when "011" => digits_out <= RD2;
104             when "100" => digits_out <= Ext_Imm;
105             when "101" => digits_out <= ALURes;
106             when "110" => digits_out <= MemData;
107             when "111" => digits_out <= WD;
108             when others => digits_out <= (others => '0');
109         end case;
110     end process;
111 end Behavioral;
```

6 Single-Cycle Execution Trace

In the first iteration, index $i = 0$, and the value read from memory is $x[0] = 0$.

Step	SW(7:5)	"000"	"001"	"010"	"011"	"100"	"101"	"110"	"111"	Only for branch	
	Instr (assembly)	Instr (hex)	PC+4	RD1	RD2	Ext_Imm	ALURes	MemData	WD	BranchAddr	JumpAddr
0	addi $1, $0, 2	20010002	00000004	00000000	00000000	00000002	00000002	XXXXXXXX	00000002	XXXXXXXX	XXXXXXXX
1	addi $2, $0, 90	2002005A	00000008	00000000	00000000	0000005A	0000005A	XXXXXXXX	0000005A	XXXXXXXX	XXXXXXXX
2	addi $3, $0, 7	20030007	0000000C	00000000	00000000	00000007	00000007	XXXXXXXX	00000007	XXXXXXXX	XXXXXXXX
3	add $4, $0, $0	00002020	00000010	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
4	add $5, $0, $0	00002820	00000014	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
5	add $6, $0, $0	00003020	00000018	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
6	slt $7, $6, $3	00C3382A	0000001C	00000000	00000007	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
7	beq $7, $0, end	10E0000F	00000020	00000001	00000000	0000000F	00000001	XXXXXXXX	XXXXXXXX	0000005C	XXXXXXXX
8	sll $9, $6, 2	00064880	00000024	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
9	add $9, $4, $9	00894820	00000028	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
10	lw $8, 0($9)	8D280000	0000002C	00000000	00000000	00000000	00000000	00000000	00000000	XXXXXXXX	XXXXXXXX
11	addi $6, $6, 1	20C60001	00000030	00000000	XXXXXXXX	00000001	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
12	slt $7, $8, $1	0101382A	00000034	00000000	00000002	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
13	bne $7, $0, loop	14E0FFF8	00000038	00000001	00000000	FFFFFFF8	00000001	XXXXXXXX	XXXXXXXX	00000018	XXXXXXXX
— *Second iteration (resume loop_start)* —											
14	slt $7, $6, $3	00C3382A	0000003C	00000001	00000007	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX

Table 3: Execution trace (First full iteration)

Part II
Pipelined Implementation

This part details the transformation of the processor into a 5-stage pipeline architecture (IF, ID, EX, MEM, WB) to improve instruction throughput by allowing multiple instructions to be processed simultaneously.

7 Pipeline Registers and Architecture

To support simultaneous instruction execution, intermediate registers were introduced between the processing stages to propagate data and control signals synchronously. The structural decision to resolve the destination register in the EX stage was implemented, moving the `RegDst` multiplexer appropriately.

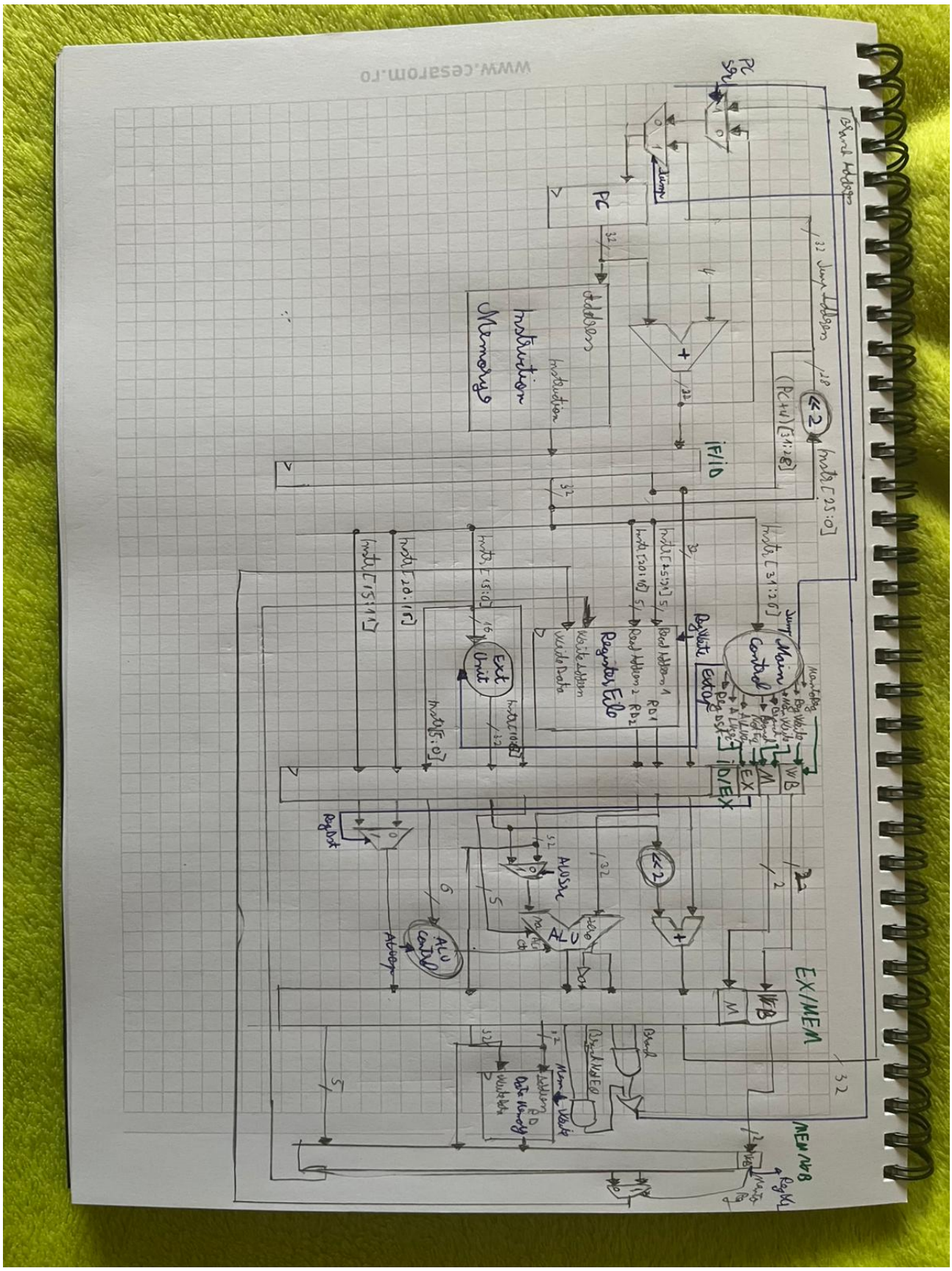

Figure 9: MIPS Pipeline Architecture Datapath

REG_IF_ID [63:0]	REG_ID_EX [157:0]	REG_EX_MEM [106:0]	REG_MEM_WB [70:0]
Instruction [63:32]	MemToReg [157]	MemToReg [106]	MemToReg [70]
PC_plus_4 [31:0]	RegWrite [156]	RegWrite [105]	RegWrite [69]
	MemWrite [155]	MemWrite [104]	MemData [68:37]
	Branch [154]	Branch [103]	ALUResout [36:5]
	BranchNotEq [153]	BranchNotEq [102]	WA [4:0]
	ALUOp [152:151]	BranchAddress [101:70]	
	ALUSrc [150]	Zero [69]	
	RegDst [149]	ALURes [68:37]	
	PC_plus_4 [148:117]	RD2 [36:5]	
	RD1 [116:85]	WA [4:0]	
	RD2 [84:53]		
	Ext_Imm [52:21]		
	func [20:15]		
	sa [14:10]		
	rt [9:5]		
	rd [4:0]		

Table 4: Bit-level mapping of the Pipeline Registers

8 Pipeline Control Signals

The control signals generated by the Control Unit are propagated synchronously through the pipeline registers. The table below details the signals for the supported instructions, including the specific `BranchNotEq` signal used for the `bne` instruction.

Instruction	Opcode [31-26]	RegDst	ExtOp	ALUSrc	Branch	Br_ne	Jump	MemWrite	MemtoReg	RegWrite	ALUOp[1:0]	function [5-0]	ALUCtrl[2:0]
add	000000	1	0	0	0	0	0	0	0	1	10	100000	000 (+)
and	000000	1	0	0	0	0	0	0	0	1	10	100100	010 (&)
slt	000000	1	0	0	0	0	0	0	0	1	10	101010	011 (<)
sll	000000	1	0	0	0	0	0	0	0	1	10	000000	100 (<<)
addi	001000	0	1	1	0	0	0	0	0	1	00 (+)	X	000 (+)
lw	100011	0	1	1	0	0	0	0	1	1	00 (+)	X	000 (+)
sw	101011	0	1	1	0	0	0	1	0	0	00 (+)	X	000 (+)
beq	000100	X	1	0	1	0	0	0	X	0	01 (-)	X	001 (-)
bne	000101	X	1	0	0	1	0	0	X	0	01 (-)	X	001 (-)
j	000010	X	X	X	0	0	1	0	X	0	XX	X	000

Table 5: Instruction encodings and control signals mapping

9 Hazard Resolution

The overlapping of instructions introduced Data Hazards (Read-After-Write) and Control Hazards (Branch/Jump). Since the architecture does not implement a hardware Forwarding Unit, hazards were resolved through software by inserting `NoOp` (No Operation) instructions into the machine code.

- **Data Hazards (RAW):** Writing to the register file occurs in the WB stage. Reading occurs in the ID stage. Up to 2 `NoOp` instructions are required to align the ID stage of a dependent instruction with the WB stage of the producer.
- **Control Hazards (Branch):** The decision for `beq` and `bne` is evaluated in the **MEM** stage. Therefore, 3 `NoOp` instructions (Delay Slots) are inserted after each conditional branch to prevent fetching incorrect instructions.
- **Control Hazards (Jump):** The target address is computed in the **ID** stage, requiring only 1 `NoOp` instruction.

10 Pipeline Execution Trace

10.1 Ideal Execution (Hazard Identification)

The table below illustrates the theoretical execution of the first loop iteration assuming no stalls. Structural (S), Data (D), and Control (C) hazards are explicitly marked in the stages where they occur, specifying the dependent instruction addresses.

Poz.	Instrucțiune	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17	C18	C19	C20	C21	C22	C23	C24	C25	C26	C27
0	addi $1, $0, 2	IF	ID	EX	MEM	WB																						
1	addi $2, $0, 90		IF	ID	EX	MEM	WB																					
2	addi $3, $0, 7			IF	ID	EX	MEM	WB																				
3	add $4, $0, $0				IF	ID	EX	MEM	WB																			
4	add $5, $0, $0					IF	ID	EX	MEM	WB																		
5	add $6, $0, $0						IF	ID	EX	MEM	**WB($6-D:6)**																	
6	slt $7, $6, $3							IF	**ID($6)**	EX	MEM	**WB($7-D:7)**																
7	beq $7, $0, end								IF	**ID($7)**	EX	**MEM(C)**	WB															
8	add $9, $4, $6									IF	ID	EX	MEM	**WB($9-D:9)**														
9	lw $8, 0($9)										IF	**ID($9)**	EX	MEM	**WB($8-D:11)**													
10	addi $6, $6, 1											IF	ID	EX	MEM	WB												
11	slt $7, $8, $1												IF	**ID($8)**	EX	MEM	**WB($7-D:12..14)**											
12	bne $7, $0, loop													IF	**ID($7)**	EX	**MEM(C)**	WB										
13	slt $7, $2, $8														IF	ID	EX	MEM	**WB($7-D:14..16)**									
14	bne $7, $0, loop															IF	**ID($7)**	EX	**MEM(C)**	WB								
15	slt $7, $0, $8																IF	ID	EX	MEM	**WB($7-D:16)**							
16	beq $7, $0, loop																	IF	**ID($7)**	EX	**MEM(C)**	WB						
17	addi $9, $8, -1																		IF	ID	EX	MEM	**WB($9-D:18)**					
18	and $9, $8, $9																			IF	**ID($9)**	EX	MEM	**WB($9-D:19)**				
19	bne $9, $0, loop																				IF	**ID($9)**	EX	**MEM(C)**	WB			
20	add $5, $5, $8																					IF	ID	EX	MEM	WB		
21	j loop_start																						IF	**ID(C)**	EX	MEM	WB	
22	j end																							IF	**ID(C)**	EX	MEM	WB

Table 6: Ideal execution highlighting structural, data, and control hazards

Poz.	Instrucțiune	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17	C18	C19	C20	C21	C22	C23	C24	C25	C26	C27	C28	C29	C30	C31	C32	C33	C34	C35	C36	C37	C38	C39	C40	C41	C42	C43	C44	C45	C46	C47	C48	C49	C50	C51	C52	C53	C54	C55	C56	C57	C58	C59	C60	C61
0	addi $1, $0, 2	IF	ID	EX	MEM	WB																																																														
1	addi $2, $0, 90		IF	ID	EX	MEM	WB																																																													
2	addi $3, $0, 7			IF	ID	EX	MEM	WB																																																												
3	add $4, $0, $0				IF	ID	EX	MEM	WB																																																											
4	add $5, $0, $0					IF	ID	EX	MEM	WB																																																										
5	add $6, $0, $0						IF	ID	EX	MEM	WB																																																									
6	NoOp							IF	ID	EX	MEM	WB																																																								
7	NoOp								IF	ID	EX	MEM	WB																																																							
8	slt $7, $6, $3									IF	ID	EX	MEM	WB																																																					
9	NoOp										IF	ID	EX	MEM	WB																																																				
10	NoOp											IF	ID	EX	MEM	WB																																																		
11	beq $7, $0, 55												IF	ID	EX	MEM	WB																																																		
12	NoOp													IF	ID	EX	MEM	WB																																																	
13	NoOp														IF	ID	EX	MEM	WB																																																
14	NoOp															IF	ID	EX	MEM	WB																																															
15	add $9, $4, $6																IF	ID	EX	MEM	WB																																												
16	NoOp																	IF	ID	EX	MEM	WB																																												
17	NoOp																		IF	ID	EX	MEM	WB																																											
18	lw $8, 0($9)																			IF	ID	EX	MEM	WB																																										
19	addi $6, $6, 1																				IF	ID	EX	MEM	WB																																									
20	NoOp																					IF	ID	EX	MEM	WB																																									
21	slt $7, $8, $1																						IF	ID	EX	MEM	WB																																								
22	NoOp																							IF	ID	EX	MEM	WB																																								
23	NoOp																								IF	ID	EX	MEM	WB																																							
24	bne $7, $0, 8																									IF	ID	EX	MEM	WB																																						
25	NoOp																										IF	ID	EX	MEM	WB																																				
26	NoOp																											IF	ID	EX	MEM	WB																																		
27	NoOp																												IF	ID	EX	MEM	WB																																
28	slt $7, $2, $8																													IF	ID	EX	MEM	WB																														
29	NoOp																														IF	ID	EX	MEM	WB																												
30	NoOp																															IF	ID	EX	MEM	WB																											
31	bne $7, $0, 8																																IF	ID	EX	MEM	WB																										
32	NoOp																																	IF	ID	EX	MEM	WB																									
33	NoOp																																		IF	ID	EX	MEM	WB																								
34	NoOp																																			IF	ID	EX	MEM	WB																							
35	slt $7, $0, $8																																				IF	ID	EX	MEM	WB																						
36	NoOp																																					IF	ID	EX	MEM	WB																					
37	NoOp																																						IF	ID	EX	MEM	WB																				
38	beq $7, $0, 8																																							IF	ID	EX	MEM	WB																			
39	NoOp																																								IF	ID	EX	MEM	WB																		
40	NoOp																																									IF	ID	EX	MEM	WB																	
41	NoOp																																										IF	ID	EX	MEM	WB																
42	addi $9, $8, -1																																											IF	ID	EX	MEM	WB															
43	NoOp																																												IF	ID	EX	MEM	WB														
44	NoOp																																													IF	ID	EX	MEM	WB													
45	and $9, $8, $9																																														IF	ID	EX	MEM	WB												
46	NoOp																																															IF	ID	EX	MEM	WB											
47	NoOp																																																IF	ID	EX	MEM	WB										
48	bne $9, $0, 8																																																	IF	ID	EX	MEM	WB									
49	NoOp																																																		IF	ID	EX	MEM	WB								
50	NoOp																																																			IF	ID	EX	MEM	WB							
51	NoOp																																																				IF	ID	EX	MEM	WB						
52	add $5, $5, $8																																																						IF	ID	EX	MEM	WB					
53	j 8																																																							IF	ID	EX	MEM	WB				
54	NoOp																																																									IF	ID	EX	MEM	WB		
55	j 55 (end)																																																										IF	ID	EX	MEM	WB	
56	NoOp																																																											IF	ID	EX	MEM	WB

Table 7: Resolved pipeline execution showing 61 clock cycles required for the first complete loop iteration

11 VHDL Code Modifications (Pipeline)

The datapath was adapted to carry signals through the intermediate pipeline registers. Port interfaces for `IDecode` and `EX` modules were updated to reflect the physical relocation of the Write Address calculation logic.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity test_env_pipeline is
    Port ( clk : in  STD_LOGIC;
           btn : in  STD_LOGIC_VECTOR (4 downto 0);
           sw  : in  STD_LOGIC_VECTOR (15 downto 0);
           led : out STD_LOGIC_VECTOR (15 downto 0);
           an  : out STD_LOGIC_VECTOR (7 downto 0);
           cat : out STD_LOGIC_VECTOR (6 downto 0));
end test_env_pipeline;

architecture Behavioral of test_env_pipeline is
    -- Component declarations (IFetch, UC, IDecode, EX, MEM, MPG, SSD)
       omitted for brevity

    -- Inter-stage signals
    signal en, rst : STD_LOGIC;
    signal Instruction, PC_plus_4 : STD_LOGIC_VECTOR(31 downto 0);
    signal PCSrc, Jump : STD_LOGIC;
    signal JumpAddress : STD_LOGIC_VECTOR(31 downto 0);
    signal RegDst, ExtOp, ALUSrc, Branch, BranchNotEq, MemWrite, MemtoReg,
       RegWrite : STD_LOGIC;
    signal ALUOp : STD_LOGIC_VECTOR(1 downto 0);
    signal RD1, RD2, Ext_Imm : STD_LOGIC_VECTOR(31 downto 0);
    signal func : STD_LOGIC_VECTOR(5 downto 0);
    signal sa : STD_LOGIC_VECTOR(4 downto 0);
    signal BranchAddress, ALURes, MemData, ALUResout : STD_LOGIC_VECTOR(31
       downto 0);
    signal Zero : STD_LOGIC;
    signal WA_out_EX : STD_LOGIC_VECTOR(4 downto 0);
    signal WD : STD_LOGIC_VECTOR(31 downto 0);

    -- PIPELINE REGISTERS
    -- 1. IF/ID
    signal IF_ID_PC_plus_4, IF_ID_Instr : STD_LOGIC_VECTOR(31 downto 0);

    -- 2. ID/EX
    signal ID_EX_PC_plus_4, ID_EX_RD1, ID_EX_RD2, ID_EX_Ext_Imm :
       STD_LOGIC_VECTOR(31 downto 0);
    signal ID_EX_func : STD_LOGIC_VECTOR(5 downto 0);
    signal ID_EX_sa, ID_EX_rt, ID_EX_rd : STD_LOGIC_VECTOR(4 downto 0);
    signal ID_EX_RegDst, ID_EX_ALUSrc, ID_EX_Branch, ID_EX_BranchNotEq,
       ID_EX_MemWrite, ID_EX_MemtoReg, ID_EX_RegWrite : STD_LOGIC;
    signal ID_EX_ALUOp : STD_LOGIC_VECTOR(1 downto 0);

    -- 3. EX/MEM
    signal EX_MEM_BranchAddress, EX_MEM_ALURes, EX_MEM_RD2 :
       STD_LOGIC_VECTOR(31 downto 0);
    signal EX_MEM_Zero, EX_MEM_Branch, EX_MEM_BranchNotEq, EX_MEM_MemWrite,
       EX_MEM_MemtoReg, EX_MEM_RegWrite : STD_LOGIC;
    signal EX_MEM_WA : STD_LOGIC_VECTOR(4 downto 0);

```

```
    -- 4. MEM/WB
    signal MEM_WB_MemData, MEM_WB_ALURes : STD_LOGIC_VECTOR(31 downto 0);
    signal MEM_WB_WA : STD_LOGIC_VECTOR(4 downto 0);
    signal MEM_WB_MemtoReg, MEM_WB_RegWrite : STD_LOGIC;

begin
    -- MPG, SSD and Module instantiations mapped to pipeline signals...

    process(clk)
    begin
        if rising_edge(clk) then
            if en = '1' then
                -- IF -> ID
                IF_ID_PC_plus_4 <= PC_plus_4;
                IF_ID_Instr     <= Instruction;

                -- ID -> EX
                ID_EX_PC_plus_4 <= IF_ID_PC_plus_4;
                ID_EX_RD1       <= RD1;
                ID_EX_RD2       <= RD2;
                ID_EX_Ext_Imm   <= Ext_Imm;
                ID_EX_func      <= func;
                ID_EX_sa        <= sa;
                ID_EX_rt        <= IF_ID_Instr(20 downto 16);
                ID_EX_rd        <= IF_ID_Instr(15 downto 11);

                ID_EX_RegDst    <= RegDst;
                ID_EX_ALUSrc    <= ALUSrc;
                ID_EX_ALUOp     <= ALUOp;
                ID_EX_Branch    <= Branch;
                ID_EX_BranchNotEq <= BranchNotEq;
                ID_EX_MemWrite  <= MemWrite;
                ID_EX_MemtoReg  <= MemtoReg;
                ID_EX_RegWrite  <= RegWrite;

                -- EX -> MEM
                EX_MEM_BranchAddress <= BranchAddress;
                EX_MEM_ALURes        <= ALURes;
                EX_MEM_RD2           <= ID_EX_RD2;
                EX_MEM_Zero          <= Zero;
                EX_MEM_WA            <= WA_out_EX;

                EX_MEM_Branch        <= ID_EX_Branch;
                EX_MEM_BranchNotEq   <= ID_EX_BranchNotEq;
                EX_MEM_MemWrite      <= ID_EX_MemWrite;
                EX_MEM_MemtoReg      <= ID_EX_MemtoReg;
                EX_MEM_RegWrite      <= ID_EX_RegWrite;

                -- MEM -> WB
                MEM_WB_MemData       <= MemData;
                MEM_WB_ALURes        <= ALUResout;
                MEM_WB_WA            <= EX_MEM_WA;

                MEM_WB_MemtoReg      <= EX_MEM_MemtoReg;
                MEM_WB_RegWrite      <= EX_MEM_RegWrite;
            end if;
        end if;
    end process;

    WD <= MEM_WB_MemData when MEM_WB_MemtoReg = '1' else MEM_WB_ALURes;
```

```
108     PCSrc <= (EX_MEM_Branch and EX_MEM_Zero) or (EX_MEM_BranchNotEq and not
            EX_MEM_Zero);
109     JumpAddress <= IF_ID_PC_plus_4(31 downto 28) & IF_ID_Instr(25 downto 0)
            & "00";
110
111 end Behavioral;
```

test_env_pipeline.vhd (Top-Level Pipeline)

Part III
Conclusions

12 FPGA Synthesis

Both architectures (Single-Cycle and Pipelined) were synthesized successfully in the Vivado environment. The resulting `.bit` files were programmed onto the FPGA board. The execution proved logically sound, correctly validating the conditional algorithms, executing conditional branches (`beq`, `bne`), arithmetic logic, and accurately accumulating the sum of valid elements while resolving inherent control and data pipeline hazards through the proper insertion of machine code NOPs.